

Gestion de code avec Git (FIEC30EM)

frank.buloup@univ-amu.fr

Aix Marseille Université
Faculté des Sciences du Sport
Master 2 IEAP - Facteurs Humains



Gestion de code avec Git

- 1 Introduction & Installation de Git
- 2 Présentation & Utilisation de Git

1 Introduction & Installation de Git

- Objectifs du module & Organisation
- les projets & Les comptes GitHub
- Les projets : c'est parti dès maintenant !
- Installation de git en local : commandes de base
- Les zones de Git

À l'issue de cette formation, vous serez capable de :

- 1 Mettre en œuvre une petite plateforme d'acquisition de données
 - 1 Utiliser Arduino IDE et un Uno R4 pour l'acquisition des données
 - 2 Utiliser Python pour créer un questionnaire d'essai
 - 3 Utiliser Python pour le traitement des données
- 2 Mettre en œuvre un projet en mode collaboratif avec Git sur GitHub
 - 1 Décrire les principes de fonctionnement d'un gestionnaire de versions distribué
 - 2 Créer, initialiser et cloner un dépôt avec Git
 - 3 Manipuler les commandes de Git pour gérer les fichiers et les branches
 - 4 Résoudre les conflits de fusion

Organisation

- Un projet par équipe pour mettre en œuvre ces apprentissages
- Cours et TDs pour apprendre les bases de Git
- Deux (à trois) étudiants par équipe

Volumes horaires

- Environ 14 heures pour le projet
- Environ 4 heures de cours/TDs

Evaluation

- 50% sur un CC Git (examen écrit 30mn)
- 50% sur le projet tout au long du module

Les projets

Sur la base d'un Arduino Uno R4 Wifi avec :

- une jauge de contrainte (2 postes)
- une résistance à détection de force (2 postes)
- un capteur à ultrason (2 postes)
- un potentiomètre
- un codeur optique

Comptes GitHub

- Création de comptes sur GitHub
- Compte du Master : MasterFHGit

Les projets - Répartition par binômes voire trinômes

Sur la base d'un Arduino Uno R4 Wifi avec :

- une jauge de contrainte (2 postes - 2x2 étudiants)
- une résistance à détection de force (2 postes - 2x2 étudiants)
- un capteur à ultrason (2 postes - 2x2 étudiants)
- un potentiomètre - 2 étudiants
- un codeur optique - 2 étudiants

[Veuillez suivre ce lien pour démarrer le projet dès maintenant](#)

[Projet] Step 1 - Synchronisation 1

- Attendre la réception d'un caractère spécial pour démarrer la boucle loop() de l'Arduino
- Ce step comportera également la configuration de la liaison série entre le PC et l'Arduino à 1Mbits/s, 8 bits de données, pas de parité et 1 bit de stop
- Vous pourriez utiliser la LED interne (LED_BUILTIN) pour vérifier que vous êtes bien entré dans la boucle

[GIT] Exercice I - Initialisation du dépôt local

- 1 Installer Git en suivant le lien suivant : [Git - SCM](#)

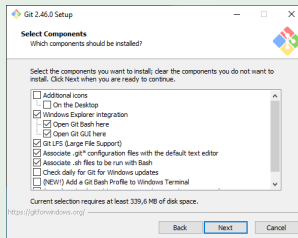


Figure: Git Setup - Garder les options par défaut

- 2 Initialiser votre dossier dans git : commande "git init"
- 3 Ajouter les fichiers à l'index : commande "git add"
- 4 Archiver cette première étape : commande "git commit"

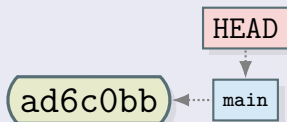
[PROJET] Step 2 - Synchronisation 2

- Dans la boucle loop(), redémarrer l'Arduino dès réception d'un caractère spécial (Software reset)
- Le redémarrage aura lieu après 10 secondes environ et sera initié côté PC

[GIT] Exercice II - Step 2 du projet

- ➊ Ajouter les fichiers à l'index : commande "git add"
- ➋ Archiver cette étape : commande "git commit"
- ➌ Visualisez l'historique avec "git log"

État initial après le premier commit local



- **ad6c0bb** est l'ID du **commit** (l'ID complet est un SHA1)
- **main** (ou **master**) est une **référence** au commit
- **HEAD** est une autre référence : celle de la "tête de lecture"

Heads

A head is simply a reference to a commit object. Each head has a name. By default, there is a head in every repository called master (or main). A repository can contain any number of heads. At any given time, one head is selected as the “current head.” This head is aliased to HEAD, always in capitals.

Note this difference: a “head” (lowercase) refers to any one of the named heads in the repository; “HEAD” (uppercase) refers exclusively to the currently active head. This distinction is used frequently in Git documentation.

[Git - Heads](#)

État après le deuxième commit local



- Il y a maintenant deux **commits**
- La **référence main** pointera toujours sur le dernier **commit**
- On voit apparaitre une **branche** qui comporte ces deux **commits**
- Cette **branche** porte le même nom que la **référence** qui pointe sur son dernier **commit** : **branche main**
- La nom **main** est par convention utilisé pour la **branche** de production, la **branche** principale
- On peut aussi trouver **master** à la place de **main** mais de moins en moins

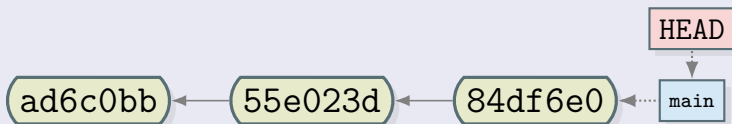
[Projet] Step 3 - Comptage temporel

- Mettre en place un système de comptage et de réinitialisation du temps en μs dans l'arduino
- La réinitialisation aura lieu à chaque redémarrage de l'Arduino
- Vous pourrez utiliser les fonctions `Serial.println()` et `delay()` pour vérifier votre code

[GIT] Exercice III - Step 3 du projet

- ➊ Ajouter les fichiers à l'index : commande "git add"
- ➋ Archiver cette. étape : commande "git commit"
- ➌ Visualisez le résumé de l'historique avec "git log --oneline"

État après le troisième commit local



- La **branche main** comporte maintenant un **commit** supplémentaire
- La **référence main** pointe donc toujours sur le dernier **commit** de sa **branche**, on ne peut pas la déplacer

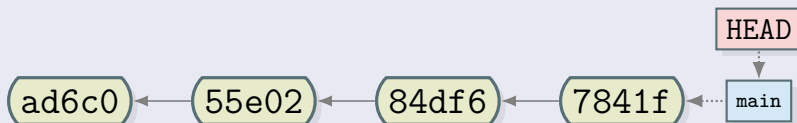
[Projet] Step 4 - Transmission du comptage temporel

- Transmettre ce comptage de temps de l'Arduino au PC
- Vous utiliserez les fonctions `sprintf()` et `Serial.println()`
- Supprimez les sorties consoles python qui prennent du temps
- Supprimez les `delay()` côté Arduino si utilisés
- Utilisez la commande `flush` du port série (Arduino et Python)
- Vous accumulerez les valeurs temporelles dans un tableau `times[]` en secondes

[GIT] Exercice IV - Step 4 du projet

- ➊ Ajouter les fichiers à l'index : commande `"git add"`
- ➋ Archiver cette. étape : commande `"git commit"`
- ➌ Visualisez le résumé de l'historique avec `"git log --oneline"`

État après le quatrième commit local

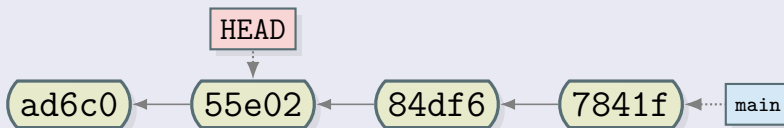


- La **branche main** comporte un autre **commit**
- La **référence main** pointe toujours sur le dernier **commit**
- La **référence HEAD**, la "tête de lecture", peut être déplacée pour explorer l'historique du dépôt

[GIT] Exercice V - Step 4 du projet : se déplacer dans l'historique

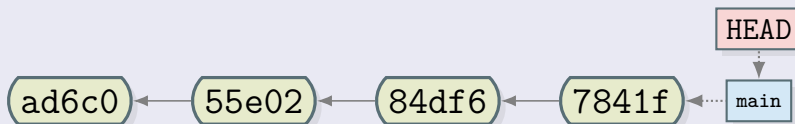
- 1 Utiliser la commande `"git checkout 55e02"` pour revenir à l'étape 2 du projet
- 2 Utiliser la commande `"git checkout main"` pour revenir en tête de la branche main

État après la commande "git checkout 55e02"



- La **référence HEAD** est déplacée au deuxième **commit**
- Les fichiers archivés se retrouvent alors dans l'état où ils étaient lors de la deuxième étape du projet
- On peut inspecter et même faire des changements
- Ils seront perdus si on déplace de nouveau **HEAD**
- Pour prendre en compte les changements, il faut créer une nouvelle **branche** et y archiver (**commit**) ces changements

État après la commande "git checkout main"



- La **référence HEAD** pointe de nouveau sur le dernier **commit** de la **branche main**
- Les fichiers archivés se retrouvent alors dans l'état le plus récent de la **branche main** et on peut continuer le travail

[Projet] Step 5 - Transmission de la valeur courante du capteur

- Lire la valeur brute du capteur (cf. doc Projets)
- Transmettre la valeur au PC
- Vous utiliserez les mêmes fonctions `sprintf()` et `Serial.println()`

Interruption inattendue : vous n'avez pas terminé cette étape et vous devez faire des modifications qui doivent être utilisées immédiatement sur le poste expérimental : changer les caractères 'r' et 's' par 'd' et 'a' pour le démarrage et l'arrêt d'un essai ! Vous ne voulez pas perdre les modifs en cours mais vous ne pouvez pas les utiliser immédiatement puisque vous n'avez pas terminé l'étape 5 ! ...

Une bonne pratique de git pourrait se résumer ainsi : "Il est souvent très utile, surtout quand un code est en production, d'utiliser une nouvelle branche lorsqu'on le fait évoluer !". Dans notre contexte, un code en production peut se traduire par "un code en cours d'utilisation dans le cadre d'une expérience, que l'on soit en phase d'acquisition ou de traitement des données".

Vous allez suivre cette bonne pratique !

[GIT] Exercice VI - Step 5 du projet : création d'une branche

- ➊ Créez une branche nommée "step5"
- ➋ Archivez les modifs en cours dans cette branche
- ➌ Retournez dans la branche main et faites les changements de caractères
- ➍ Archivez ces changements et utilisez-les en poste !
- ➎ Retournez dans la branche "step5" et terminez l'étape

[GIT] Exercice VII - Step 5 du projet : fusion de branches

Pour appliquer les améliorations de l'étape 5 dans la branche principale (main) :

- ➊ Vous retournez dans la branche main
- ➋ Vous fusionnez la branche "step5" dans cette branche

Petit résumé intermédiaireire

Concepts importants à retenir

- Branches
- Commits
- Heads (références, pointeurs)
- Toute branche a son pointeur de même nom qui référence toujours le dernier commit de la branche
- HEAD est le seul pointeur qui peut bouger. C'est à partir de lui que sera créé le prochain commit.
- La fusion de branches peut être une opération délicate si de trop nombreux changements ont été effectués aux mêmes endroits dans les deux branches

Petit résumé intermédiaireire

Quelques Commandes : status, log, branch et switch

```
1 $ git status           # Git status
2 $ git log              # Print commits
3 $ git log --oneline     # Print abbrev-commits
4 $ git branch branchName # Create branch branchName
5 $ git switch branchName # Move HEAD to branchName
6 $ git switch -c bName   # Create and switch to bName
7 $ git merge bName       # Merge bName in current branch
```

Il existe un répertoire .git

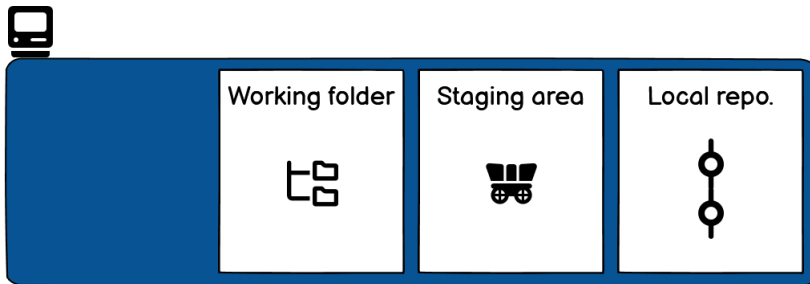
- Il contient la base de donnée locale du dépôt : dépôt local
- Il contient un fichier de configuration du dépôt

Petit résumé intermédiaireire

I need help !

```
1 $ git commit --help # open html or man page on "commit"
2 $ git remote -h      # Quick "remote" help in console
```

Les zones de Git



2 Présentation & Utilisation de Git

- Git ? C'est quoi ?
- Petit historique des systèmes de contrôle de versions
- Les différents modes d'utilisation



Une idée ?

[Documentation Git](#)

Git n'est pas un serveur (au sens centralisé)

- Git n'offre pas de fonctionnalité de gestion de projet complète (e.g. gestion de tâches, de bugs ou wiki)
- Github et GitLab sont deux exemples de solutions complètes



Github



GitLab



Bitbucket



Gitea

Git n'est pas un serveur (au sens centralisé)

- Git n'offre pas de fonctionnalité de gestion de projet complète (e.g. gestion de tâches, de bugs ou wiki)
- Github et GitLab sont deux exemples de solutions complètes



Github



GitLab



Bitbucket



Gitea

Git n'est pas un espace de stockage de fichiers type Dropbox

- Pas de sauvegarde automatique des fichiers
- Pas de gestion optimale des fichiers volumineux* ou binaires**

*Dans ce cas, on peut utiliser une extension spécifique : git LFS

**Les fichiers binaires ne sont pas très lisibles (git diff !!!)

Git est un système de contrôle de version (Version Control System)

- Historique des versions de tous les fichiers
- Manipulation complète de l'historique possible
- Décentralisé (bientôt plus clair...)
- On trouve aussi : "Source Code Management System"

Git est un système de contrôle de version (Version Control System)

- Historique des versions de tous les fichiers
- Manipulation complète de l'historique possible
- Décentralisé (bientôt plus clair...)
- On trouve aussi : "Source Code Management System"

Pourquoi gérer des versions de fichiers, utiliser un VCS ?

- Pour pouvoir revenir en arrière
- Pour comparer des versions de fichiers
- Pour expérimenter, tester des idées
- Pour travailler à plusieurs plus facilement

Vous pouvez gérer tous types de fichiers avec git !

Exemples de fichiers gérés par git

- Fichiers de code informatique
- Fichiers textes (e.g. .doc)
- Fichiers binaires (e.g. images, sons) - Si volumineux : git LFS*
- Fichiers de données - Si volumineux : git LFS*
- Documentation de projets (cahier des charges, devis...)
- etc.

Gestion collaborative de projets

*Utilisation de l'extension [Git Large File Storage](#)

Taille max. de 5GB sur Github (AWS Amazon S3)

Un peu d'histoire !

Local VCS - Avant... il y a longtemps !

- Système D : horodatage, classement dossiers, compression etc.
- Pro : Revision Control System (1982, projet GNU*)



Work in progress !

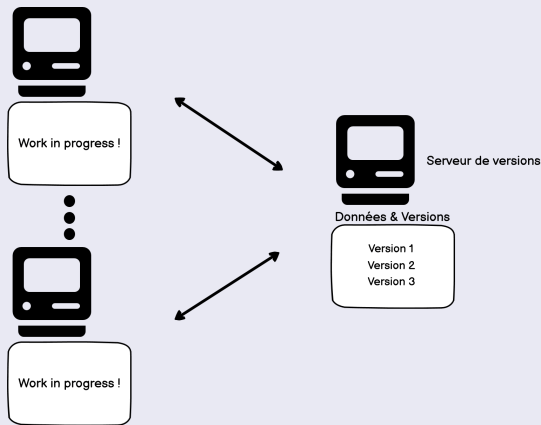
Archives

Version 1
Version 2
Version 3

* GNU : GNU's Not Unix

Centralized VCS - Avant... un peu moins longtemps ! 90's

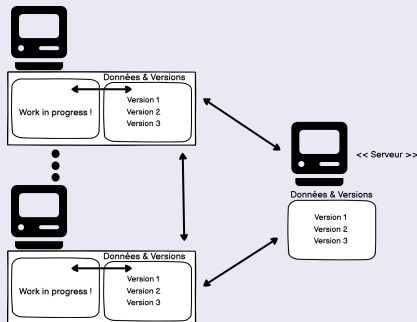
- Partage plus facile
- Connexion réseau obligatoire (approche Client-Serveur)



e.g. Concurrent Version System (CVS), Subversion (SVN)

Distributed VCS - Maintenant, depuis un moment (début 2000) !

- État "serveur" et réseau pas critique (approche Pair-à-Pair)
- On peut remonter un "serveur" à partir des autres machines
- Les opérations locales (la majorité) sont rapides
- Travail privé possible, sans serveur



e.g. Git, Mercurial, Darcs

L'origine de Git

- Créé en 2005 par [Linus TORVALDS](#)
- Pour le noyau Linux
- Activement maintenu par [Junio HAMANO](#)
- Popularisé par GitHub
- On prononce "guit" et pas "jit" !
- Git = Crétin (= Conna... !)

[Git - Wikipedia](#)

Les principales caractéristiques de Git

- Contrôle de version distribué (pair-à-pair)
- Gestion des branches de code (création, suppression, fusion)
- Opérations atomiques (Commits)
- Données de gestion stockée dans un répertoire .git

Comment utiliser Git ?

- En ligne de commande (Terminal & Git Bash pour Windows)
- Outils graphiques intégrés à Git : "git-gui" et "gitk"
- Avec une IHM dans un OS (GitKraken, SourceTree) : [ici](#)
- Intégré dans un EDI (plugin, package etc.)
- Sur internet : github, gitlab, bitbucket etc.



GitKraken



SourceTree

L'apprentissage en ligne de commande est le plus efficace !